

CS349: Course Project

XData Extensions

Project Report

Satyankar Chandra (22B0967) Ayan Tanwar (22B0931)
Harsh Kumar (22B0973) Gourish Garg (22B0915)

Spring 2025

Contents

1	Introduction	2
2	Project Goals	3
3	Main Contributions	3
3.1	User Experience Features	3
3.1.1	Course Creation or Selection 1a	3
3.1.2	Loading and Modifying Databases 1b	4
3.1.3	Assignment Creation and Deployment 2a, 2b	4
3.1.4	Query Playground 3a, 3b	4
3.1.5	Query Planner 4b, 4a	4
3.1.6	Syntactic Constraints on Queries	5
3.2	Utility Features	6
3.2.1	Database Viewer 5	6
3.2.2	Query History Tracker 6a	6
3.2.3	Exporting into different formats 6b	7
3.2.4	Version Control for Databases	7
4	Implementation Details	8
4.1	Technology Stack	8
4.2	Frontend Functionality	8
4.3	Backend Functionality	8
5	Integration with XData	9
6	Programming Methodology	9
7	Future Work	9
8	Conclusion	9
8.1	Summary	10
8.2	Acknowledgements	10

Source Code: <https://github.com/SuperSat001/CS349-Course-Project/>

Guide: Prof. S. Sudarshan and Prof. Suraj Shetiya
Dept. of Computer Science and Engineering, IIT Bombay

1 Introduction

This report documents the work done in the CS349 course project during the Spring 2024 - 2025 semester. Our project topic is **XData Extensions**, where we aim to extend the functionality of the XData system.

During this project, our team has implemented several features and improvements to the existing XData platform. The goal of this project is to enhance the usability and functionality of the XData system, making it more user-friendly for both students and faculty members.

For demonstration purposes, we have created a simple web application that showcases the new features our team has developed in isolation. This application serves as a minimal working example of our work, and code for the same is available in the GitHub repository linked above. We have also ensured that our features are compatible with the existing XData system. At our backend, we have used the same database schema as XData to store data about students, courses, and assignments.

Our team has spent a considerable amount of time and effort on this project throughout the semester. The rough timeline of our work is as follows:

- **Week 1-2:** Setting up the project, **creating a small web application which uses PGLite** to host a local in-browser database. Deciding on the features to be implemented.
- **Week 3-5:** Implementing features for **Query Planning, Syntax Constrained Querying and Multi-database setup** in PGLite. Improving the ReactJS frontend to provide a better user experience for these features.
- **Week 6-7:** Integrating the **current XData schema** to store assignments and evaluated data. Implementing UI for instructors to create courses and assignments, and for students to attempt them. Implementing a **Query Playground** for students to test their queries and get feedback.
- **Week 8:** Implementing other QoL improvements such as **Query History**, a simple **version control system** for loaded databases, enhancing **Import/Export** functionality, and other minor features.
- **Week 9:** Finalizing the project, UI improvements, basic security testing and bug fixing.

Around Week 6, we also submitted a mid-semester report and part of the code to the instructors. We also met with them to discuss our progress and get crucial feedback on our work. The feedback was very helpful in guiding us towards the final implementation - especially with the XData integration (using the same database schema) as well as some of our core features. Based on the feedback, we discarded some of our initial ideas and focused on the features that would be most useful for students and instructors and are easier to integrate with the existing XData system.

Full details of the features we have implemented are discussed in section 3. We have also included in section 7 a list of improvements and features that we would like to implement given more time in the future.

2 Project Goals

The main problem statement we started with was as follows:

XData Extensions Problem Statement

Extend XData to support in-browser postgresql execution using <https://pglite.dev>. Make sure everything runs from local resources so it can be run for exams without external internet. This allow each user to run in their own database without accessing any central database. The next step is to extend XData system to run the queries in browser rather than on a backend database.

With this in mind, our team started brainstorming ideas for features to implement. We tried to code the features in isolation, and then integrate them into a single web application. This helped us to work on the features independently and in parallel, while also ensuring that they would work together in the final product.

We started with the following features as a part of our initial project proposal:

- Evaluation Plans and Query Optimization
- Query Profiling and Runtime Analysis
- Support for Concurrent Queries and Multiple Databases
- Adding Syntactic Constraints to Queries
- Improvements to the XData UI to better support the new features

After discussing with the instructors, we received positive feedback on our implementation of the Query Planning features and frontend UI. However, it was evident that some of our initial ideas (on Query Profiling) were not feasible within the scope of the project and it would be better to focus on the remaining features along with the their integration with the current XData system. After getting access to the XData GitLab and the database schema, we spent most of our time during the second half on the integration aspect of the project.

The final set of implemented features is presented in the next section.

3 Main Contributions

In this section, we will discuss the main features we have implemented in our project. These features are roughly listed in the order of typical user interaction. We also have a few other features which are not directly related to user interaction, but are important for the overall functionality of the system.

3.1 User Experience Features

3.1.1 Course Creation or Selection [1a](#)

We allow instructors to modify assignments/schema of existing courses or create new courses. Changes to a course are automatically reflected in the database with the XData schema.

(a) Adding a Course

(b) Loading a Database

3.1.2 Loading and Modifying Databases [1b](#)

Both instructors and students can load databases from local files or by manually entering the schema. The databases are stored in the browser's local storage via PGLite. The databases are fully isolated from each other, and users can switch between them anytime without any issues. Tuples can be added, modified or deleted from the database at any time.

3.1.3 Assignment Creation and Deployment [2a](#), [2b](#)

After selecting a course and loading the database schema, instructors can create assignments for students. The assignments are again stored in the XData schema, and can be modified or deleted at any time. Instructors can supply a valid SQL query for each question in the assignment, which will be used to evaluate the students' submissions. Students can then attempt the assignments using the Query Playground.

3.1.4 Query Playground [3a](#), [3b](#)

The Query Playground is a convenient interface for students to test their queries and get instant feedback. The playground allows students to run any SQL query on the loaded database, and provides a simple interface to view the results. Students can load their own data with the same schema, and compare the results with the expected output.

3.1.5 Query Planner [4b](#), [4a](#)

PostgreSQL typically uses a cost-based query planner to determine the best execution plan for a given query. However, it does not allow users to specify their own query plans. We have implemented this functionality by setting and unsetting particular flags so that query is forced

Edit Assignment (Course: bruh | ID: 1)

Assignment Details

Assignment Name*

Demo Assignment

Description

Hello, world!

Base Student Schema Definition*

schema_1 (ID: 1)

Database Connection*

(None)

(a) Editing an Assignment

Questions

Question 1

Question Text*

Select all students with ID 12345

Correct SQL Query*

select * from student where id = '12345';

Marks*

1

☒ Order Independent?

Text, query, and non-negative marks required.

(b) Making a Question

Query Playground (Assignment 1 / Question 1)

[Back to Assignment](#)

Select all students with ID 12345

Your SQL Query:

select * from student where id = '12345';

☒ Order Matters

[Run Query & Compare](#)

✓ Results Match!

Your Output

id	name	dept_name
12345	Shankar	Comp. Sci.

Correct Output

id	name	dept_name
12345	Shankar	Comp. Sci.

(a) Right Answer

Query Playground (Assignment 1 / Question 1)

[Back to Assignment](#)

Select all students with ID 12345

Your SQL Query:

select * from student where id = '12';

☒ Order Matters

[Run Query & Compare](#)

✗ Results Do Not Match.

Your Output

(Query returned no rows)

Correct Output

id	name	dept_name
12345	Shankar	Comp. Sci.

(b) Wrong Answer

to run in a particular way. This allows users to experiment with different query plans and see how they affect the performance of the query.

3.1.6 Syntactic Constraints on Queries

Often, instructors want to enforce certain syntactic constraints on the queries submitted by students. For example, they may want to ensure that students do not use certain keywords or functions in their queries. We tried to implement this feature by using a simple parser on the query string and whitelisting/blacklisting certain keywords. However, this approach becomes overly restrictive for complex constraints and hence this feature is still a work in progress.

Query Plan:

```
Index Scan using student_id_index on student  (cost=0.15..8.17 rows=1 width=152)
  Index Cond: ((id)::text = '12'::text)
Planning Time: 0.100 ms
Execution Time: 0.100 ms
```

Executed Command:

```
EXPLAIN ANALYZE select * from student where id = '12';
```

Scan Preference Options

Current Scan Preference: **indexscan**

(a) Forcing Index Scan

Query Plan:

```
Seq Scan on student  (cost=0.00..15.62 rows=1 width=152) (actual time=0.000..0.000 rows=0 loops=1)
  Filter: ((id)::text = '12'::text)
  Rows Removed by Filter: 13
Planning Time: 0.100 ms
Execution Time: 0.000 ms
```

Executed Command:

```
EXPLAIN ANALYZE select * from student where id = '12';
```

Scan Preference Options

Current Scan Preference: **seqscan**

(b) Forcing Sequential Scan

3.2 Utility Features

3.2.1 Database Viewer [5](#)

We have implemented a simple database viewer which allows users to view and edit the contents of the database. The viewer displays the tables in the database, along with their schema and data. Users can also add, modify or delete tuples from the database using this interface. This feature is useful for users who want to quickly view the contents of the database without having to run any queries.

3.2.2 Query History Tracker [6a](#)

We have implemented a simple query history tracker which displays the last 10 queries run by the user in a side panel. There is a button to copy the query to the clipboard, so that the user can easily reuse it. This feature is useful for users who want to quickly access their previous queries without having to retype them.

Database Browser (Schema: schema_1)

Select Table from 'schema_1'

advisor

Insert Row into schema_1.advisor

Enter s_id

Enter i_id

Insert

Delete Row from schema_1.advisor

Enter value for primary key: s_id

Delete by Primary Key

Displaying Rows from schema_1.advisor (Max 100)

Reload Rows

s_id	i_id
00128	45565
12345	10101
23121	76543
44553	22222
45678	22222
76543	45565

Figure 5: Database Viewer

Query History:

select * from student where id = '12';

select * from student; Click to copy

select * from student;

(a) Showing Query History

Export Table to CSV

schema_1_takes

Convert to CSV

(b) Export Table to CSV

3.2.3 Exporting into different formats [6b](#)

We allow users to export the database into easy-to-read formats such as CSV, JSON, and also as a SQL dump. This allows users to easily share their databases with others, or to import them into other systems.

3.2.4 Version Control for Databases

We have implemented a simple version control system for the databases. Users can create a new version of the database at any time, and switch between different versions. This allows users to easily revert to a previous version of the database if needed. This feature is still a work in progress, and we plan to improve it further in the future.

4 Implementation Details

The key components of our code are implemented using the suggested PGLite library to ensure complete in-browser execution and compatibility with the current XData system. We have taken care to ensure that our code is modular, well-documented, easy to understand, and easy to integrate with the existing XData system. We have also tried to avoid any external dependencies to ensure that our code can run in a completely offline environment.

4.1 Technology Stack

We have used the following technology stack for our project:

- **Frontend:** ReactJS, CSS
- **Build Tool:** Vite
- **Backend:** PGLite

4.2 Frontend Functionality

The frontend of our web application is built using ReactJS and CSS. We have implemented a smooth and responsive UI which allows the users to easily access all the features we have developed. Key highlights of the frontend include:

- A simple and intuitive interface for users to load databases (by importing schema and data files), modify them, and run queries.
- Isolated tabs for each feature and a prominent navigation bar to switch between them.
- A Query Playground for students to test their queries and get instant feedback.
- Query History shown in a side panel, which allows users to easily access and retry their previous queries.
- Comprehensive error messages and debugging information printed to the console.

4.3 Backend Functionality

The backend of our web application is built using PGLite, which allows us to run PostgreSQL queries in the browser. The database is stored in the browser's local (indexedDB) storage, which allows us to run queries without any external internet access. The key features of the backend include:

- Complete support for all PostgreSQL queries, including option to EXPLAIN/ANALYZE.
- Support for loading multiple databases (or versions of the same database) and switching between them. Complete isolation of databases to prevent any corruption.
- Fully local execution of queries, with no external dependencies and a lightweight footprint.

5 Integration with XData

Our frontend components for managing courses, assignments, and questions were designed to interact with tables mirroring the core XData schema (`xdata_course`, `xdata_assignment`, `xdata_qinfo`), ensuring data compatibility. We utilized the `xdata_schemainfo` table, which allowed us to register and manage multiple dynamically loaded student database schemas within the PGLite environment, associating them with specific courses while maintaining the standard XData metadata tables within the `public` PostgreSQL schema path. This approach preserves structural alignment with XData while facilitating the offline, in-browser functionality central to our project goals.

6 Programming Methodology

Due to the nature of our project, we had to work in parallel on the frontend and backend components. We used Git/GitHub for version control so that all team members could work on the same codebase in different branches without any conflicts.

We used LLM based tools to help us efficiently generate frontend code (mostly the styling and UI components). We observed that these tools made the development process much faster with us providing the high-level UI/UX design and the LLMs generating CSS. However, this was only possible for the frontend code (with simple components) and not for the backend code. The backend code was much more complex (especially keeping in mind the XData integration and the database schema) and required a lot of manual effort from our side to implement and debug.

7 Future Work

We have implemented a lot of features in our project, but there is definitely room for improvement.

Some avenues for future work include:

- Adding support for more types of assignments with complex grading schemes.
- Allowing interaction between different databases (e.g. joining tables etc.)
- Adding support to change the database schema of an existing database.
- Integrating a better text editor for writing queries, with support for syntax highlighting and auto-completion.
- Improving the Syntax Constraints feature to allow more complex constraints and better error messages.
- Created a more robust version control system at individual transaction level with options to manually commit or rollback changes.

8 Conclusion

We feel that have made significant progress in our project on XData Extensions. Our team has successfully implemented several features and improvements to the existing XData system, and we have an intuitive and user-friendly web application to showcase our work.

Our team is very proud of the work we have done and we hope that our project will be useful for perhaps future iterations of the XData system. We are really interested to continue working on this project and fully integrate the completed features into the XData.

8.1 Summary

In this project, we have implemented several novel features including a Query Planner, Query Playground, Syntactic Constraints on Queries. We have created a web application from scratch using ReactJS and PGLite, which allows users to run PostgreSQL queries fully in-browser. We have also added functionality to easily import, export and modify multiple databases which work in full isolation. We have also implemented a bunch of QoL improvements such as Query History Tracker, Version Control for databases, and a convenient UI for instructors to create courses and assignments. We have also integrated some of our features with the existing XData system, and ensured that our code is modular and easy to understand.

8.2 Acknowledgements

We would like to thank our instructors, Prof. S. Sudarshan and Prof. Suraj Shetiya, for their guidance and support throughout the project. Their feedback (both over email and in-person) was invaluable in helping us shape our project and implement the best set of features. We are all really grateful for their time spent with us and prompt responses to all our queries. We are also thankful to the whole XData team who built and maintain the XData system, which provided us with a great platform to build upon. We would also like to thank our peers in the course for their support and feedback during the project.

Signing off,
Satyankar, Ayan, Harsh and Gourish